Planul proiectului, feature cu feature

Spring Boot 3.5 · Java 21 · MySQL 8.4 · Flyway · Spring AI 1.1 (MCP) · Kafka · Prometheus + Grafana · Tempo + ELK · Keycloak · Kong · doar Docker, pe EC2 · Sebi

Pornești de la codul tău, nu de la zero. Ai deja piesele grele: `BatteryCommandValidator` și `applyCommand` (fizica bateriei, cu randament), `EnergyEngine` (bilanțul casei), `AgentMode` (cele 4 niveluri de autonomie), `SimulationProperties`, `GridProperties`, contul `AGENT` din `UserType` și un CI verde cu 4 joburi. Le lipsesc legăturile dintre ele. Planul acesta le leagă, pe rând.
Înlocuiește `PLAN_BACKEND.pdf` și `FEATURES.pdf`. Tabelele rămân în `STRUCTURA_BD.pdf`, contractul REST în `openapi.yaml`. Bug-urile din `CODE_REVIEW.pdf` (runda 8) se repară **în feature-ul căruia îi aparțin**: le găsești la „Repară”.
Scopul: să construiești singur tot procesul unui sistem real: microservicii, Spring AI prin MCP, logare, metrici, tracing, Kafka, un server de identitate (Keycloak) și un API gateway (Kong), fiecare intrat în proiect la momentul în care îți rezolvă o problemă. Totul pe **Docker** (docker compose pe EC2), fără Kubernetes.
Fiecare feature merge de la A la Z, în aceeași ordine:
ce poate face utilizatorul → migrare `Vn__…sql` → model + repository → service → intrare (REST, tool MCP sau Kafka) → permisiuni → test cu login real → log / metrică → verde în Actions → verificat pe EC2
Un feature e gata doar când trece „Gata când” **pe EC2, cu token din `/auth/login`**. Nu începi următorul înainte. Fiecare feature aduce **cel mult o unealtă nouă**, marcată **nou**.

Unde ajungi

```
Internet → nginx (TLS) → Kong (F15) → solar-sync-broker-api ── MySQL
                    │                          │ MCP (F7–F8)
                    │            solar-sync-broker-agent (F9–F10) → model AI
                    └ oauth2-proxy → Grafana · Kibana · Kafka UI
simulator (F12) → Kafka (F11) → api → Kafka → agent
Keycloak (F14) emite tokenurile · Prometheus + Grafana (F6)
Tempo + Elasticsearch / Kibana / Filebeat (F13) peste toate
```

Diagrama completă, cu rețelele Docker și profilurile compose: `ARHITECTURA.pdf`, lângă acest plan.

Cine ce poate — enum-ul complet, de la F1

Permisiune	USER (proprietar)	OPERATOR	ADMIN	AGENT	Folosită în
`HOUSE_VIEW`	casa lui	toate	toate	toate	F2, F7
`HOUSE_MANAGE`	–	–	✓	–	F2
`BATTERY_VIEW`	a lui	toate	toate	toate	F3, F7
`BATTERY_CONFIG`	a lui	–	✓	–	F3
`BATTERY_COMMAND`	a lui	toate	toate	–	F3
`TELEMETRY_VIEW`	a lui	toate	toate	toate	F4, F7
`FINANCIAL_VIEW`	a lui	toate	toate	–	F5
`COMMUNITY_VIEW`	–	✓	✓	✓	F4, F5, F7
`GRID_VIEW`	–	✓	✓	✓	F6, F7
`SIMULATION_CONTROL`	–	–	✓	–	F4
`AGENT_VIEW`	–	✓	✓	✓	F8
`AGENT_PROPOSE`	–	–	–	✓	F8
`AGENT_APPROVE`	–	✓	✓	–	F8
`USER_MANAGE`	–	–	✓	–	F1

„a lui” = permisiunea e în token, iar service-ul verifică proprietarul. `AGENT` nu are `BATTERY_COMMAND`: agentul doar **propune**, nu comandă direct (F8). `BATTERY_MANAGE` dispare. De la F14 rolurile vin din Keycloak; harta rol → permisiuni rămâne în API, aceeași.

I. Comunitatea merge

O singură aplicație, `solar-sync-broker-api`. La final ai simularea, banii și rețeaua, toate vizibile în Grafana.

F1 Contul meu

vizitator · utilizator · administrator

Vizitatorul își face cont de proprietar și se loghează. Oricine logat își vede contul și permisiunile. Administratorul creează conturi OPERATOR, USER și AGENT.

Pornești de la: `AuthController`, `AuthServiceImpl`, `JWTTokenProvider`, `UserController`, `UserPermissions`, `UserType` · **Repară: B2, B9** · **nou** Flyway, Testcontainers, failsafe

1. **Bază.** `V1__schema.sql` creează cele 6 tabele exact cum le cer entitățile tale la finalul F1 (`validate` nu te lasă să minți). Restul corecturilor vin fiecare cu feature-ul ei, în migrarea ei: tabelul „Migrările" din `STRUCTURA_BD.pdf`. `ddl-auto: validate`. Baza `solar_db` de pe EC2 o ștergi o singură dată: datele sunt generate.
2. **Permisiuni.** Enum-ul complet din tabelul de mai sus și o singură hartă `UserType` → permisiuni, citită de `getAuthorities()`.
3. **Service.** `register` face doar `USER`, orice ar trimite clientul. Un `ApplicationRunner` creează contul `ADMIN` din `ADMIN_EMAIL` / `ADMIN_PASSWORD`, dacă nu există. `addUser` salvează parola prin `PasswordEncoder` și tipul.
4. **Intrare.** `POST /api/v1/auth/register`, `/auth/login`, `GET /auth/me`, `POST /api/v1/users` (`USER_MANAGE`).
5. **Test.** `maven-failsafe-plugin`, iar CI-ul rulează `mvn -B verify`. Testele de integrare pe MySQL din Testcontainers, cu `@ServiceConnection`; H2 iese. Un helper de test face register → login → token; **de aici, toate testele pe endpoint-uri protejate îl folosesc**, nu `@WithMockUser`.
6. **Livrare.** `JWT_SECRET_KEY`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` stau în `.env` pe EC2, niciodată în repo.

Reguli

- Email valid și unic (409); parolă de minim 8 caractere; vârstă ≥ 18. Login greșit → 401, același mesaj pentru email inexistent și parolă greșită.
- Parola nu apare în niciun răspuns. Fiecare șir din `hasAuthority(...)` există în enum și îl primește cine trebuie.

Gata când: register cu `"userType":"ADMIN"` → contul creat e `USER` · `/auth/me` fără token → 401 · `POST /users` ca USER → 403 · pe EC2: login cu adminul din `.env` → `/auth/me` = 200.

F2 Casa cu bateria ei

administrator · operator · proprietar

Administratorul adaugă, modifică și șterge case; bateria apare odată cu casa. Proprietarul își vede casa; operatorul și administratorul le văd pe toate.

Pornești de la: `HouseController`, `HouseCommandServiceImpl`, `HouseQueryServiceImpl`, `HouseRepository`, `Battery`, `GlobalExceptionHandler` · **Repară: B1, B5, B8**

1. **Bază.** `V2__battery_capacity.sql`: `batteries.capacity_kwh`.
2. **Service.** `createHouse` salvează casa și bateria ei într-o singură tranzacție. Ștergerea unei case cu istoric → 409.
3. **Proprietarul, într-un singur loc.** O metodă care decide accesul la o casă: `ADMIN` și `OPERATOR` trec, proprietarul se verifică în SQL (`existsByIdAndOwner_Email`), restul primește excepția de acces. F3, F4, F5 și F7 o refolosesc.
4. **Erori.** O singură excepție de acces, mapată pe 403. Handler-ele întorc `ApiErrorResponse` întreg, nu un câmp din el.
5. **Intrare.** Cele 6 operații din `openapi.yaml`, cu `HOUSE_VIEW` / `HOUSE_MANAGE`.
6. **Test.** Matricea cu trei conturi reale: admin, proprietar, străin, pe fiecare endpoint.

Reguli

- Nume obligatoriu și unic (400 / 409); puterile > 0 (400); proprietar inexistent → 404.
- Bateria implicită: 10 kWh, SoC 50%, 3 kW încărcare / 3 kW descărcare, randament 95%.
- Casa cu istoric nu se șterge (409); o scoți din simulare cu `enabled: false`.

Gata când: ADMIN creează casa lui X → X o vede, cu bateria ei · Y → 403 cu corp complet · `/houses/999` → 404 cu corp complet · ștergerea unei case cu comenzi → 409.

F3 Comand bateria — poarta

proprietar · operator · administrator

Proprietarul își vede bateria, îi schimbă puterile maxime, îi trimite `CHARGE`, `DISCHARGE` sau `IDLE` cu puterea în kW și vede istoricul comenzilor.

Pornești de la: `BatteryCommandValidatorImpl`, `applyCommand`, `executeCommand`, `CommandExecution`, `BatteryController` · **Repară: B3, B4, B6**

1. **Bază.** `V3__commands.sql`: pe `command_executions`, `target_soc` → `power_kw`, plus `failure_reason`, `soc_after_percent`, `executed_by` mai lung și cheia străină pe `house_id`.
2. **Fizica, într-un singur loc.** Capacitatea vine din baterie, nu din constantă. Limitele 10 și 90 stau într-un singur loc, citit și de validator, și de `applyCommand`.
3. **O singură cale de execuție.** `executeCommand(houseId, command, executedBy)`: verifică accesul (F2), validează, aplică, salvează comanda. **F8 și F9 o refolosesc; nu se scrie alta.**
4. **Service.** Request-ul are `powerKw` cu `@Positive`, nu `targetSoc`. Prinzi doar cele două excepții ale validatorului → `REJECTED` cu motivul; orice altă eroare nu e `REJECTED`. `updateConfiguration` scrie în bază și verifică proprietarul.
5. **Intrare.** `getBattery`, `updateConfiguration`, `sendCommand`, `getCommands`.
6. **Test.** Fiecare test de comandă verifică **SoC-ul de după și statusul**, nu doar codul HTTP.

Reguli

- O comandă acționează un tick (o oră virtuală). Peste puterea maximă sau cu SoC-ul ieșit din 10–90% → se salvează `REJECTED` (`BATTERY_POWER_LIMIT` / `BATTERY_SOC_LIMIT`), iar bateria nu se schimbă.

Gata când: proprietarul: `CHARGE` 2 kW pe SoC 50, 10 kWh, 95% → 201 `SUCCESS`, SoC 69 · `CHARGE` 90 kW → 201 `REJECTED` `BATTERY_POWER_LIMIT`, SoC tot 69 · PATCH 4.5 kW → GET arată 4.5 · străinul → 403.

F4 O zi simulată

administrator · proprietar · operator

Cu profilul `demo`, la prima pornire există 10 case cu proprietarii lor. Administratorul rulează un tick: fiecare casă activă primește producție PV, consum, preț și un rând de telemetrie. Proprietarul își vede ultimele 50 de măsurători; operatorul vede starea comunității.

Pornești de la: `EnergyEngineImpl`, `SimulationProperties`, `ClockConfig`, `Telemetry`, `PvGenerator` (comentat), `LoadGenerator` și `SimulationClock` (goale), `TelemetryRepository` (comentat), `ComunityStateResponse` · **nou** logare structurată

1. **Bază.** `V4__telemetry.sql`: `recorded_at` și `price_per_kwh` pe telemetrie, cheie unică pe `(house_id, recorded_at)`.
2. **Calcul pur.** `SimulationClock` ține ora virtuală și o avansează cu o oră la fiecare tick. `PvGenerator`, `LoadGenerator`, `PriceGenerator`: din sămânță, oră și casă → o valoare. Fără bază, fără Spring în calcul; teste unitare pe fiecare.
3. **Service.** `SimulationService.tick()`: pentru fiecare casă activă PV + consum → `EnergyEngine.compute` → `Telemetry`; un singur `saveAll`. Până la agent, bateriile stau `IDLE`. `DemoDataSeeder` cu `@Profile("demo")`, care nu face nimic dacă există deja case.
4. **Intrare.** `POST /api/v1/simulation/tick` (`SIMULATION_CONTROL`), `GET /houses/{id}/telemetry`, `GET /community/state`.
5. **Logare.** Fiecare tick primește un `tickId`. `tickId` și `houseId` intră în MDC, deci apar pe fiecare linie de log. O linie `INFO` per tick (câte case, cât a durat), `DEBUG` per casă. În profilul de pe EC2, logurile ies ca JSON (`logging.structured.format.console`).

Reguli

- Aceeași sămânță și aceeași oră → aceleași valori. Noaptea PV = 0; la prânz, aproape de `pvPeakPowerKw`.
- Un tick = o oră virtuală. Telemetria casei altcuiva → 403. A doua pornire nu mai creează nimic.

Gata când: 10 tick-uri → 100 de rânduri · repornit pe bază goală cu aceeași sămânță → aceleași valori · în logurile de pe EC2 filtrezi un singur tick după `tickId`.

F5 Banii casei

proprietar · operator

La fiecare tick, importul casei se înregistrează ca `BUY`, exportul ca `SELL`, la prețul orei. Proprietarul își vede tranzacțiile și totalul; operatorul vede comunitatea.

Pornești de la: `FinancialCommandServiceImpl.recordTransaction` (fără apelant), `FinancialController`, `FinancialQueryServiceImpl`, `FinancialControllerIntegrationTest` · **Repară: B7** · **nou** `BigDecimal` pe bani

1. **Bază.** `V5__financial.sql`: sumele pe `DECIMAL`, `recorded_at`, `house_id` ca cheie străină, cheie unică pe `(house_id, recorded_at)`.
2. **Service.** Tick-ul cheamă `recordTransaction` cu prețul din telemetria aceluiași tick. Valoarea = kWh × preț, pe `BigDecimal`.
3. **Intrare.** Pe `/api/v1`: `/houses/{houseId}/financial-summary` (proprietarul, prin metoda din F2) și `/community/financial-summary`.
4. **Test.** `FinancialControllerIntegrationTest` pornește de la login real; dacă trece, permisiunile chiar există.

Gata când: după un tick, suma `total_value` la `BUY` pe o casă = importul ei × prețul din telemetrie · străinul → 403 · pe EC2 cu token real → 200.

F6 Starea rețelei

operator · administrator

La fiecare tick se calculează încărcarea transformatorului comun și se salvează. Operatorul vede starea; în Grafana vede comunitatea live.

Pornești de la: `GridProperties` (50 kW, marjă 10%) · **nou** Micrometer + Prometheus + Grafana

1. **Bază.** `V6__grid_snapshots.sql`.
2. **Calcul pur.** `GridGuard`: din importul și exportul total + `GridProperties` → stare și kW în plus. Teste pe cele trei praguri.
3. **Service + intrare.** Tick-ul salvează un rând. `GET /api/v1/grid/state` (`GRID_VIEW`).
4. **Metrici.** Actuator + `micrometer-registry-prometheus`. Gauge-uri: import și export total, starea rețelei, SoC mediu. Contoare: comenzi pe status, tick-uri. Timer: durata tick-ului. `/actuator/prometheus` se vede doar din rețeaua compose.
5. **Ops.** Prometheus și Grafana în compose; dashboard-ul (JSON) și sursa de date în `solar-sync-broker-ops/grafana/`, încărcate automat la pornire.

Reguli

- Limita = 50 kW × (1 − 10%) = 45 kW. Import și export separat: ≤ 45 `OK`, până la 50 `WARNING`, peste `CRITICAL`, cu kW-ii în plus.

Gata când: comunitatea împinsă artificial peste 50 kW → `/grid/state` = `CRITICAL` și panoul din Grafana se face roșu · după 24 de tick-uri, graficul SoC-ului mediu arată ziua.

II. MCP — piesa centrală

Comunitatea devine ceva ce poate folosi un agent: întâi doar citind, apoi propunând. Primul microserviciu apare aici, firesc.

F7 Comunitatea văzută de un agent

orice client MCP

Un client MCP (MCP Inspector, Claude Desktop, mai târziu brokerul tău) se conectează cu un token și întreabă de case, baterii, telemetrie, prețuri, rețea și bani. Vede exact cât vede utilizatorul tokenului.

Pornești de la: query service-urile din F2–F6 · **nou** server MCP (Spring AI 1.1, `spring-ai-bom` e deja în `pom.xml`)

1. **Dependență.** Starter-ul de server MCP pe WebMVC, transport Streamable HTTP.
2. **Tool-uri, în pachetul `mcp/`.** `listHouses`, `getHouse`, `getBattery`, `getTelemetry(houseId, last ≤ 50)`, `getCommunityState`, `getGridState`, `getPriceHistory(hours ≤ 24)`, `getFinancialSummary(houseId)`.
3. **Regula tool-urilor.** Un tool = un apel la un service existent. Fără repository, fără logică în tool. Descrierea tool-ului și a fiecărui parametru e scrisă pentru cine o citește: un model.
4. **Permisiuni.** Endpoint-ul MCP stă în spatele aceluiași filtru JWT. Verificarea proprietarului rămâne în service, deci e aceeași prin REST și prin MCP.
5. **Log / metrică.** Fiecare apel: numele tool-ului și utilizatorul, într-o linie de log și un contor per tool.

> **Gata când:** MCP Inspector cu token de OPERATOR listează cele 8 tool-uri, iar `getGridState` dă același rezultat ca REST · cu tokenul unui proprietar, `getTelemetry` pe casa altcuiva → eroare de acces · fără token → 401.

F8 Propun → aprob → execut

agent · operator

Un agent propune un plan: o decizie per casă (`CHARGE` / `DISCHARGE` / `IDLE`, kW, motiv). Ce se întâmplă cu planul decide `AgentMode`. Orice execuție trece prin poarta din F3.

Pornești de la: `AgentMode`, `AgentProperties`, `executeCommand` (F3), `BatteryCommandValidator`, `CommandExecution` · **nou** tool-uri MCP de scriere

1. **Bază.** `V7__agent.sql`: `agent_runs`, `agent_decisions`, `command_executions.agent_decision_id`. Contul `AGENT` îl creează administratorul, prin `POST /users`.
2. **Service.** Propunerea salvează run-ul și fiecare decizie, trecută prin validator **fără să atingă bateria**: `valid` + `rejection_reason`. Aprobarea cheamă `executeCommand` din F3 pentru fiecare decizie validă, cu `executed_by` = agentul + cine a aprobat.
3. **Modurile.** `DISABLED`: propunerea e refuzată · `ADVISORY`: doar se salvează · `SUPERVISED`: așteaptă aprobarea unui om · `AUTONOMOUS`: se execută imediat, tot prin poartă.
4. **Intrare.** Tool-uri MCP `proposePlan(decisions)` (`AGENT_PROPOSE`) și `getRun(id)`. REST: `GET /api/v1/agent/runs`, `POST /api/v1/agent/runs/{id}/approve` (`AGENT_APPROVE`).
5. **Test.** Contul `AGENT` propune; operatorul aprobă; agentul care încearcă să aprobe singur → 403.

> **Gata când:** din MCP Inspector, cu token `AGENT`, în `SUPERVISED`: un plan cu o decizie de 2 kW și una de 90 kW → după aprobare: o comandă `SUCCESS` cu `agent_decision_id`, iar decizia de 90 kW rămâne neexecutată, cu motivul.

F9 Brokerul pe reguli — primul microserviciu

agent

O aplicație separată, `solar-sync-broker-agent`, care la fiecare oră virtuală citește comunitatea prin MCP, aplică regulile tale și propune planul. Nu are bază de date și nu cunoaște API-ul REST, doar tool-urile.

Pornești de la: regulile din `Descrierea Simplă a Proiectului.pdf`, `AgentProperties.planningIntervalSeconds`, contul `AGENT` · **nou** client MCP, a doua aplicație

1. **Proiect.** Folderul `solar-sync-broker-agent/`: Boot 3.5, starter-ul de client MCP. La pornire se loghează ca `AGENT` (credențiale din `.env`) și trimite tokenul la serverul MCP.
2. **Calcul pur.** Interfața `AgentPlanner` și `RuleBasedPlanner`, câte o metodă pe regulă, cu teste unitare pe fiecare (în lucrare, fiecare devine un paragraf).
3. **Bucla.** La interval: tool-uri de citire → planner → `proposePlan`. Fiecare rulare are un `runId` în MDC.
4. **Ops.** Dockerfile propriu, serviciu nou în compose, job nou în CI, filtrat pe `paths`.

> **Reguli**
> - Pragul = media prețurilor din ultimele 24 de ore virtuale. Preț < 0,75 × prag și SoC < 90 → `CHARGE` la puterea maximă. Preț > 1,4 × prag și SoC > 20 → `DISCHARGE` la puterea maximă. Altfel `IDLE`.
> - Rețeaua `WARNING` / `CRITICAL` are prioritate: descărcările se reduc până când exportul total intră sub 45 kW.

> **Gata când:** pe EC2, cu ambele aplicații pornite: după un tick, în `/agent/runs` apare un plan · aprobat în `SUPERVISED` → SoC-urile se schimbă · agentul oprit → API-ul merge mai departe.

F10 Brokerul cu AI

agent

Același agent primește un al doilea planner, cu un model AI. Alegi din configurare care rulează. Rezultatul: comparația reguli vs. AI, pe aceeași zi simulată, pentru lucrare.

Pornești de la: `AgentPlanner` și clientul MCP din F9 · **nou** Spring AI `ChatClient`

1. **Planner.** `LlmPlanner`: `ChatClient` cu tool-urile MCP din F7 (modelul își cere singur datele) și ieșire structurată → lista de decizii. Prompt-ul stă într-un fișier din `resources`, cu limitele și scopurile comunității.
2. **Siguranță.** Timeout. Dacă modelul nu răspunde sau răspunde greșit, rulează `RuleBasedPlanner` și se scrie un `WARN`. Cheia API stă în `.env`.
3. **Metrici.** Tokenii și latența apelurilor ajung în Prometheus, lângă celelalte.
4. **Comparația.** Aceeași sămânță, 24 de tick-uri, fiecare planner pe rând: profit, minute în `CRITICAL`, % decizii respinse de poartă, latență și cost.

> **Gata când:** tabelul comparației e completat din bază și din Grafana · fără cheie API, agentul trece pe reguli și o spune în log.

III. Evenimente și servicii

Comunitatea trece de la „cine cheamă pe cine” la „cine ascultă ce”. La final ai tot drumul în aceeași aplicație distribuită: servicii, AI, evenimente și observabilitate, legate între ele.

F11 Tick-ul anunță, ceilalți reacționează

Pornești de la: tick-ul din F4, banii din F5, rețeaua din F6, agentul din F9 · **nou** Kafka

1. **Ops.** Kafka în compose, un singur nod (KRaft), în profilul `messaging`; topicurile le creează un container `kafka-init`, nu codul. Deploy-ul pe EC2 pornește doar profilurile care încap acolo.
2. **Evenimente.** După commit, tick-ul publică `telemetry.recorded` (per casă, cu `tickId`); rețeaua publică `grid.state`.
3. **Ascultători, tot în monolit.** Banii și rețeaua devin `@KafkaListener` în loc de apeluri directe din tick. Agentul pornește la `grid.state`, nu la timer.
4. **Idempotență.** Același eveniment primit de două ori nu dublează nimic (cheie unică pe casă + oră).

> **Gata când:** tick → tranzacțiile apar prin listener · același eveniment trimis de două ori → o singură tranzacție · listener oprit și repornit → recuperează ce a pierdut.

F12 Simulatorul pleacă din monolit

Pornești de la: generatoarele și `SimulationClock` din F4 · **nou** topicurile ca singur contract între servicii

1. `solar-sync-broker-simulator`: doar generatoarele și ceasul. Fără bază. Publică măsurătorile (PV, consum, preț) per casă și oră.
2. API-ul le consumă: aplică bateria, `EnergyEngine`, salvează telemetria și publică mai departe, ca în F11.

> **Gata când:** API-ul fără simulator merge, doar fără tick-uri · cu aceeași sămânță, telemetria e identică cu cea din F4.

F13 Urmăresc un tick prin tot sistemul

Pornești de la: logurile din F4 și metricile din F6 · **nou** OpenTelemetry + Tempo + ELK (Elasticsearch, Kibana, Filebeat)

1. **Trace-uri.** OpenTelemetry în toate cele trei aplicații, propagat prin headerele Kafka și prin apelurile MCP, trimis prin OTLP în Tempo; în Grafana vezi drumul unei cereri.
2. **Loguri.** Filebeat citește logurile JSON ale containerelor și le trimite în Elasticsearch; în Kibana cauți după `tickId` sau `traceId`. `traceId` intră în MDC, deci apare pe fiecare linie.
3. **Ops.** Profilurile `obs` (Tempo) și `logs` (Elasticsearch, Kibana, Filebeat). Elasticsearch cere 1–2 GB: local sau pe instanța de demo.

> **Gata când:** cauți un `tickId` în Kibana, iei `traceId`-ul și în Tempo vezi un singur drum: simulator → Kafka → API → rețea → agent → MCP → aprobare → comandă.

IV. Platforma, ca la cars

Ultimele două piese ale unei platforme reale: identitatea scoasă din aplicație și o singură poartă de intrare. Tot pe Docker.

F14 Cine ești, spus de Keycloak

toți · agentul · administratorul

Conturile și parolele nu mai stau în API. Keycloak emite tokenurile: pentru oameni (login), pentru agent (fără parolă) și pentru UI-urile de administrare.

Pornești de la: matricea de permisiuni din F1, `UserType`, `SecurityConfiguration`, `JWTTokenProvider`, contul `AGENT` din F8–F9 · **nou** Keycloak (OAuth2 / OIDC)

1. **Ops.** Keycloak și Postgres-ul lui în compose, profilul `auth`. Pornește cu `--import-realm`, din `realm-solar-sync.json` pus în `solar-sync-broker-ops/keycloak/`. Parola de admin și secretul clientului agentului: doar în `.env`.
2. **Realm.** Roluri `USER`, `OPERATOR`, `ADMIN`, `AGENT`. Clienți: `solar-sync-api`, `solar-sync-agent` (client credentials, cu rolul `AGENT`) și `oauth2-proxy`.
3. **Bază.** `V8__users_keycloak.sql`: `password` dispare, apare `keycloak_id`. Utilizatorul local rămâne, pentru că o casă are proprietar: la prima cerere cu un token nou, API-ul îl creează după `sub` și email.
4. **API.** Devine OAuth2 resource server pe issuer-ul realm-ului. Rolurile din token trec prin aceeași hartă rol → permisiuni din F1. `/auth/register`, `/auth/login` și `JWTTokenProvider` ies.
5. **Agentul.** Își ia tokenul prin client credentials, nu prin login cu parolă.
6. **UI-uri.** oauth2-proxy în fața Grafana, Kibana și Kafka UI: login prin Keycloak, doar grupul `/admins`.
7. **Test.** Testele cu login real iau tokenuri de la un Keycloak din Testcontainers, pornit cu același realm.

Reguli

- Niciun rol nu se mai alege la înregistrare: contul de proprietar îl face Keycloak, rolurile le dă administratorul în realm.
- Matricea de permisiuni nu se schimbă; se schimbă doar cine emite tokenul.

Gata când: cu token din Keycloak, proprietarul → casa lui 200, casa altuia 403 · agentul, cu client credentials → `proposePlan` merge, aprobarea → 403 · vechiul `/auth/login` → 404 · Grafana fără login → pagina Keycloak; un utilizator din afara `/admins` → 403.

F15 O singură poartă: Kong

toți

Aplicațiile nu mai sunt expuse direct. Tot traficul spre ele, inclusiv al agentului, trece printr-un API gateway care verifică tokenul, limitează abuzul și pune un id pe fiecare cerere.

Pornești de la: compose-ul din `solar-sync-broker-ops/`, tokenurile din F14, MDC-ul din F4 · **nou** Kong (API gateway) + rețele Docker

1. **Ops.** Kong DB-less în compose, cu `kong.yml` declarativ în `solar-sync-broker-ops/kong/`. Rute: `/api/v1` și `/mcp` → API. În fața lui, nginx doar pentru TLS (Let's Encrypt).
2. **Plugin-uri.** `jwt` cu cheia publică a realm-ului din F14 · `rate-limiting` · `correlation-id` (header-ul ajunge în MDC și în trace) · `prometheus` (metricile lui Kong în Grafana).
3. **Rețele Docker.** `edge`: nginx, Kong, oauth2-proxy · `backend`: aplicațiile, Kafka, MySQL, Keycloak. Doar Kong și oauth2-proxy sunt în amândouă. API-ul, agentul și simulatorul nu mai au `ports:`.
4. **Agentul.** Apelează MCP prin Kong, nu direct.

Reguli

- Pe EC2 sunt deschise doar 80 și 443 (și 22, doar de la IP-ul tău). Orice alt port publicat e o greșeală.
- Kong verifică tokenul, dar permisiunile rămân în API: poarta nu ia decizii de domeniu.

Gata când: portul API-ului, încercat direct pe EC2 → nu răspunde · prin Kong fără token → 401 de la Kong, cu token → 200 · peste limita de cereri pe minut → 429 · id-ul de corelare din răspuns îl găsești în Kibana și în Tempo.

Toate cele 15 fac parte din proiect, în ordinea asta. Fiecare se sprijină pe cel dinainte: nu se sare. Dacă susținerea vine înainte de F15, în lucrare intră ce merge pe EC2 la data aceea.
Rămâi pe Spring Boot 3.5 și pe Spring AI 1.1 până la susținere (Spring AI 2.x cere Boot 4). Exemplele din proiectele pe Boot 4 nu se copiază 1:1.
Doar Docker, cu profiluri compose (`core`, `messaging`, `obs`, `logs`, `auth`, `edge`). Pe EC2-ul obișnuit: `core` + `obs`, apoi `auth` + `edge`. Kafka, simulatorul și Elasticsearch cer o instanță mai mare: local sau pe instanța de demo pentru susținere.
Secretele (`JWT_SECRET_KEY` până la F14, parolele, secretul clientului agentului, cheia API a modelului) stau doar în `.env`, pe EC2 și local. Niciodată în repo, niciodată în compose.